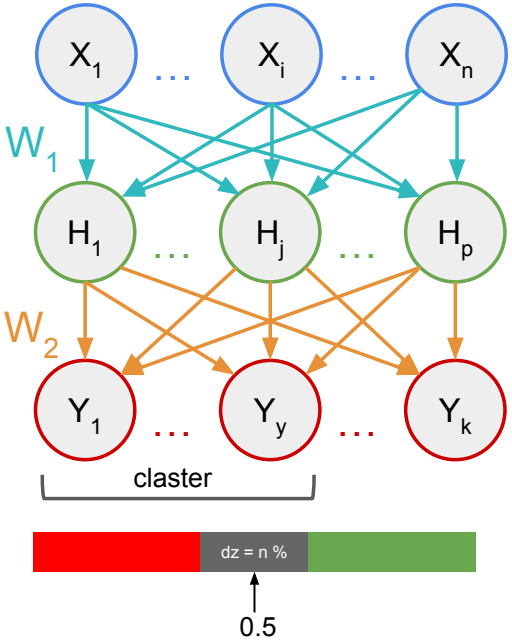
Secure Split-Neural Synchronization

On Secure Neural Synchronization: A Teacher-Student Key Exchange Protocol with Homomorphically Encrypted Channel and SHA-256 Verification

Structure

Teacher (Server)

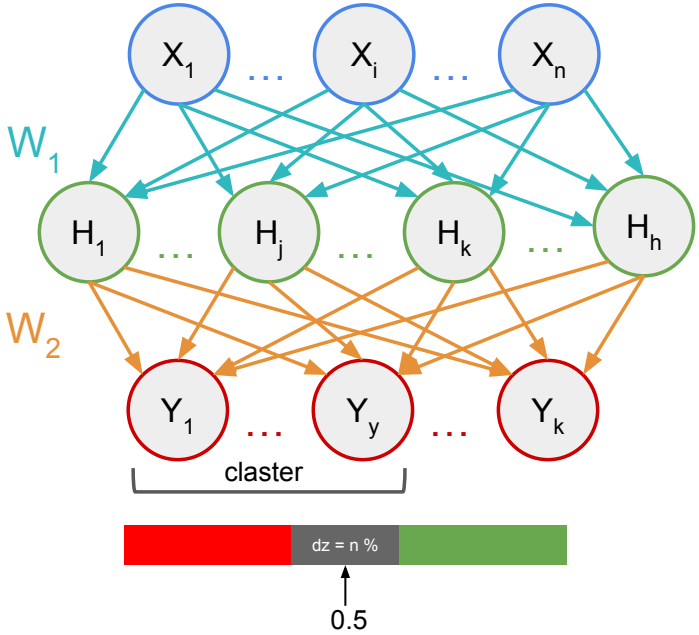


Initialization

Synchronization

Key Derivation

Student (Client)



Initialization

Teacher (Server)

Target Function:

$$W_{1,T},\ W_{2,T} \in \mathbb{R}^{d_{in} \times d_{hid\_T}} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{d_{in}}}\right)$$

Feedback Alignment Matrix:

$$B_{FA} \in \mathbb{R}^{d_{out} \times d_{hid\_S}} \sim \mathcal{N}\left(0, \frac{1}{\sqrt{d_{out}}}\right)$$

Student (Client)

Client State:

$$W_{1,S},\ W_{2,S} \in \mathbb{R}^{d_{in} \times d_{hid\_S}} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{d_{in}}}\right)$$

Cryptographic Context (FHE)

FHE Parameters:

- ***N*** = 4096
- **Modulus *chain*** = *[60, 40, 40, 60]* bits
- **Δ** = 2^{40}

Cryptographic Keys:

- ***sk:*** Sparse ternary polynomial (Secret, Client-only).
- ***pk:*** Public Key polynomial pair ***(b, a)*** (Server-visible).
- ***evk:*** Evaluation Key RNS-gadget polynomials (Server-visible).

Synchronization

Teacher (Server)

$$Y_{true} = \mathrm{ReLU}(X \cdot W_{1,T}) \cdot W_{2,T} \in \mathbb{R}^{B \times d_{out}}$$
$$Conf = 2 \cdot |\sigma(Y_{true}) - 0.5| \quad \in [0,1]$$
$$Y_{eff} = Y_{true} \cdot (1 + \alpha \cdot Conf)$$
$$[[Error]] = \frac{[[Y_{pred}]] - Y_{eff}}{B}$$
$$[[\nabla W_{2,S}]] = [[H_S]]^T \cdot [[Error]]$$
$$[[E_{hid}]] = [[Error]] \cdot B_{FA}$$

$X,\ [[H_S]],\ [[Y_{pred}]]$

$[[\nabla W_{2,S}]],\ [[E_{hid}]]$

Student (Client)

$$X \sim \mathcal{N}(0,1) \in \mathbb{R}^{B \times d_{in}}$$
$$H_{pre} = X \cdot W_{1,S}$$
$$H_S = \mathrm{ReLU}(H_{pre})$$
$$Y_{pred} = H_S \cdot W_{2,S}$$
$$[[H_S]] = Enc_{pk}(H_S);\ [[Y_{pred}]] = Enc_{pk}(Y_{pred})$$

$$\begin{cases} [[x]] = \mathrm{Enc}_{pk}(x) \\ [[\mathrm{y}]] = \mathrm{Enc}_{pk}(y) \end{cases}$$

$$\nabla W_{2,S} = Dec_{sk}([[\nabla W_{2,S}]]);\ E_{hid} = Dec_{sk}([[E_{hid}]]);$$
$$E_{corr} = E_{hid} \odot \mathrm{ReLU}'(H_{pre})$$
$$\nabla W_{1,S} = X^T \cdot E_{corr}$$
$$\mathrm{clip\_L2}(\nabla W_1, 1.0), \quad \mathrm{clip\_L2}(\nabla W_2, 1.0)$$
$$W_{1,S} \leftarrow \mathrm{Adam}(W_{1,S}, \nabla W_1, st\_W_1, lr_t)$$
$$W_{2,S} \leftarrow \mathrm{Adam}(W_{2,S}, \nabla W_2, st\_W_2, lr_t)$$

Key Derivation

$X_{pub} \sim \mathcal{N}(0,1) \in \mathbb{R}^{1 \times d_{in}}$

X_{pub} →

$Y_T = \sigma(ReLU(X_{pub} \cdot W_{1,T}) \cdot W_{2,T}) \in \mathbb{R}^{d_{out}}$

$Y_S = \sigma(ReLU(X_{pub} \cdot W_{1,S}) \cdot W_{2,S}) \in \mathbb{R}^{d_{out}}$

Cluster & Dead-Zone Filtering

$$\mu_{k,T} = \frac{1}{|C_k|} \sum_{i \in C_k}^{N} Y_{i,T}$$

$$\mathcal{B}_T(\mu_{k,T}) = \begin{cases} 1, & \text{if } \mu_{k,T} \geq 0.5 + dz \\ 0, & \text{if } \mu_{k,T} \leq 0.5 - dz \\ \perp, & \text{otherwise} \end{cases}$$

$$Idx_T = \{(k,\ \mathcal{B}_T(\mu_{k,T})) \,|\, \mathcal{B}_T(\mu_{k,T}) \neq \perp\}$$

Idx_S ↔ Idx_T

$$\mu_{k,S} = \frac{1}{|C_k|} \sum_{i \in C_k}^{N} Y_{i,S}$$

$$\mathcal{B}_S(\mu_{k,S}) = \begin{cases} 1, & \text{if } \mu_{k,S} \geq 0.5 + dz \\ 0, & \text{if } \mu_{k,S} \leq 0.5 - dz \\ \perp, & \text{otherwise} \end{cases}$$

$$Idx_S = \{(k,\ \mathcal{B}_S(\mu_{k,S})) \,|\, S_S(\mu_{k,S}) \neq \perp\}$$

Hash-based Verification

$Shared\_Idx = Idx_S \cap Idx_T$

$h_S = \text{SHA-256}(\mathcal{B}_S\,[Shared\_Idx])$

h_S ↔ h_T

$Shared\_Idx = Idx_S \cap Idx_T$

$h_T = \text{SHA-256}(\mathcal{B}_T\,[Shared\_Idx])$

If $h_S == h_T \implies$ ACCEPT (Session key confirmed)

If $h_S \neq h_T \implies$ REJECT (Retry with new X_{pub})

Adam Optimization

Core Concept: Adam (Adaptive Moment Estimation) computes individual, dynamic learning rates for each parameter. By maintaining exponential moving averages of the gradients (momentum) and their squared values (variance), it successfully neutralizes the high-variance cryptographic and structural noise produced by the Feedback Alignment process, ensuring fast and stable convergence.

1. Moving Averages (Moments):

$$\begin{cases} m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \\ v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \end{cases}$$

2. Bias Correction:

$$\begin{cases} \hat{m}_t = \frac{m_t}{1-\beta_1^t} \\ \hat{v}_t = \frac{v_t}{1-\beta_2^t} \end{cases}$$

3. Parameter Update:

$$W_t = W_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

State variables:

t - current time step(epoch)
g_t - current gradient
m_t - momentum (moving average gradients)
v_t - variance (variance estimate)

Constants:

η_t - learning rate
β_1 - “memory” coefficient for momentum
β_2 - “memory” coefficient for variance
ε - to avoid the "divide by zero" error

L2 Gradient Norm Clipping

Limits the **L2-norm** of weight gradients to a threshold (1.0). If the norm exceeds 1.0, the vector is scaled down to unit length.

- **Impact:** Ensures update stability in Adam Optimization.

$$\text{clip\_L2}(\nabla W_1, 1.0), \quad \text{clip\_L2}(\nabla W_2, 1.0)$$

Math logic

$$||\nabla W||_2 = \sqrt[2]{\sum_i^N (\nabla W_i)^2}$$

$$\text{if } ||\nabla \text{W}|| > 1.0 \Rightarrow \nabla W_{clipped} = \nabla \text{W} \cdot \frac{1.0}{||\nabla W||_2}$$

Homomorphic encryption

- ***Enc***$_{pk}$: The encryption function that secures a plaintext value using a public key
- ***Dec***$_{sk}$: The decryption function that restores the original value using a secret key

$$\begin{cases} [[x]] = \mathrm{Enc}_{pk}(x) \\ [[\mathrm{y}]] = \mathrm{Enc}_{pk}(y) \end{cases}$$

- The essence of Homomorphic Encryption is the ability to perform mathematical operations directly on encrypted data without ever exposing the actual numbers to the server

$$\begin{cases} \mathrm{Dec}_{sk}([[x]] \oplus [[y]]) = x + y \\ \mathrm{Dec}_{sk}([[x]] \otimes [[y]]) = x \cdot y \end{cases} \qquad \begin{cases} \mathrm{Dec}_{sk}([[x]] \oplus c) = x + c \\ \mathrm{Dec}_{sk}([[x]] \otimes c) = x \cdot c \end{cases}$$

System Parameters

$$R_q = \mathbb{Z}_q[X]/\langle X^N + 1 \rangle$$

- **Polynomial Degree (N):** N = 4096 (must be a power of 2 for FFT algorithm)
- **RNS Modulus Chain (q):** $q = q_0 \cdot q_1 \cdot q_2 \cdot q_3$
  - **Bit-lengths:** [60, 40, 40, 60] (≈ 200 bits total).
- **Scaling Factor (Δ):** 2^{40}

Key Generation (Client Side Only)

- **Secret Key (s):** Sample a sparse ternary polynomial $s \leftarrow R_q \ \ with \ \ s_i \in \{-1, 0, 1\}$
  - *Parameter:* Hamming weight ≈ 64 (not 0)

- **Public Key (pk = (b, a)):** Sample $a \leftarrow R_q$ (uniform) and error $e \leftarrow \chi_{err}$ (Gaussian).
  - *Compute:* $b = -a \cdot s + e \pmod{Q}$

- **Evaluation Key (evk):** RNS-gadget decomposed encryptions of s^2.
  - *Compute:* For each *RNS* prime q_i, compute $b_i = -a_i \cdot s + e_i + s^2 \cdot q_i^{-1}$

Encoding & RNS Lifting

IFFT & Scaling: Transform a complex vector into a polynomial

- **Goal:** Converts a complex vector *m* (raw data, e.g., ML model weights) into a polynomial plaintext suitable for lattice-based encryption.
- **IFFT (Inverse FFT):** Maps the continuous complex numbers from the slot domain into the polynomial coefficient domain.
- **Scaling (Δ):** Multiplies values by a large factor **Δ** *(e.g., 2^{40})* to preserve precision before rounding.
- ***Compute**:* $coeff_j = \lfloor \Delta \cdot \mathrm{IFFT}(m)_j \rceil$

RNS Lifting: Decompose the massive integer polynomial into the Residue Number System.

- ***Compute**:* Create arrays $P_0, P_1, P_2, P_3 \ where \ P_i = Poly \pmod{q_i}$

Encryption

Ephemeral Masking: Sample a single-use ternary blinding polynomial u $\leftarrow R_q$

- **BV-style Encryption:** Sample fresh small Gaussian errors e_0, e_1
  - $c_0 = b \cdot u + e_0 + plaintext \pmod{Q}$

    $c_1 = a \cdot u + e_1 \pmod{Q}$

- ***Result**:* Ciphertext tuple (c_0, c_1).

Homomorphic Evaluation

Addition: Component-wise addition in RNS.

- $(c_0, c_1) + \left(c_0', c_1'\right) = (c_0 + c_0', c_1, + c_1') \pmod{q_i}$

Multiplication (Tensor Expansion):

- *Compute:* $d_0 = c_0 c_0'$, $d_1 = c_0 c_1' + c_1 c_{0,}' d_2 = c_1 c_1'$
- *State:* Ciphertext expands to a triplet (d_0, d_1, d_2). Scale becomes $\mathbf{\Delta^2}$

Relinearization: Restore the triplet back to a pair using *evk*.

- *Compute:* Slnew\ice d_2 across RNS moduli, multiply by corresponding evk_i components, and accumulate into d_0, d_1.
- *Result:* $\left(c_0, c_{new\_1}\right)$

Rescaling (Modulus Drop): Manage the squared scale $\left(\Delta^2 \rightarrow \Delta\right)$

- *Compute:* Divide the ciphertext by the last modulus q_{last} using RNS basis extension (multiply by q_{last}^{-1} (*mod* q_i) for remaining *i*).
- *State:* Multiplicative depth decreases by 1 level.

Decryption & Decoding

RNS Decryption: Extract the noisy message.

- *Compute:* $m_{poly} = c_0 + c_1 \cdot s \pmod{q_i}$

CRT Reconstruction (Garner's Algorithm): Lift from RNS to Big Integer.

- *Compute:* Reconstruct a 256-bit integer *X* from the remaining RNS components using mixed-radix conversion:

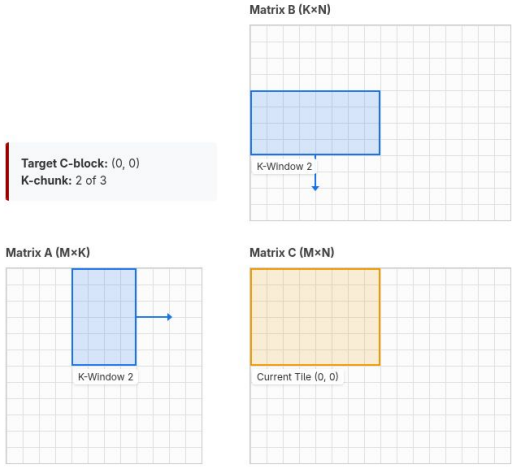
  $X = v_0 + v_1 q_0 + v_2(q_0 q_1) + v_3(q_0 q_1 q_3)$

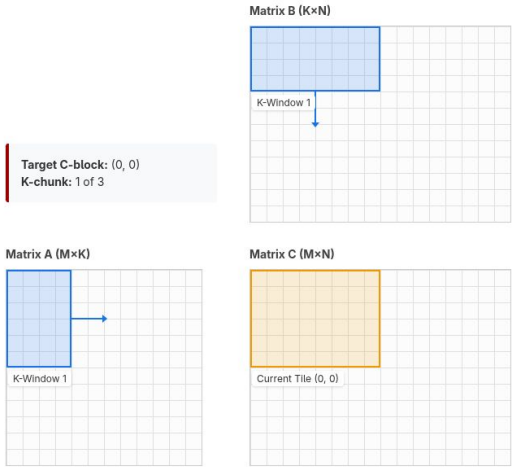
Decoding: Convert polynomial back to complex vector.

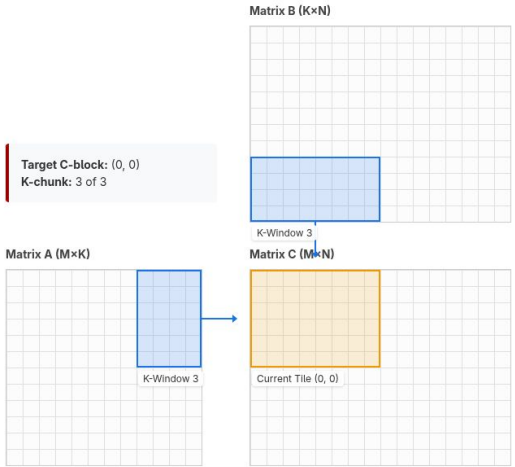
- *Compute:* $Divide\,by\,current\,\Delta \rightarrow\ Forward\,FFT \rightarrow Extract\,original\,data$

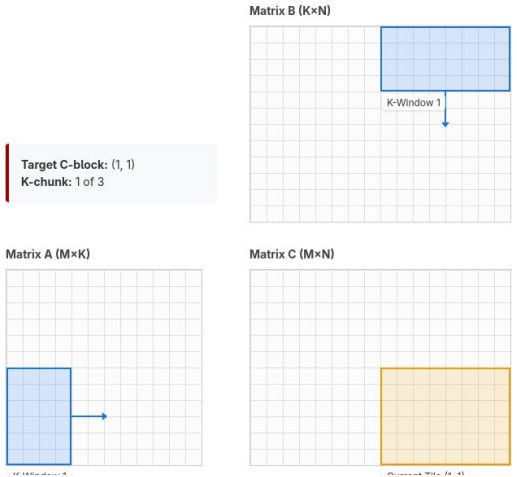
Block Matrix Multiplication

Instead of processing massive matrices all at once, the algorithm divides them into smaller, fixed-size sub-matrices (blocks or tiles) and computes the final result piece by piece.









FFT

$$X[k] = \sum_{n=0}^{N/2-1} x[2n] e^{-i \frac{2\pi}{N/2} kn} + W_N^k \sum_{n=0}^{N/2-1} x[2n+1] e^{-i \frac{2\pi}{N/2} kn}$$

IFFT

$$x[k] = \frac{1}{N} \left(\sum_{n=0}^{N/2-1} X[2n] e^{+i \frac{2\pi}{N/2} kn} + W_N^{-k} \sum_{n=0}^{N/2-1} X[2n+1] e^{+i \frac{2\pi}{N/2} kn} \right)$$